

Sparkbills

Multi-Industry GST Billing & ERP SaaS —
Product Overview + Full Architecture

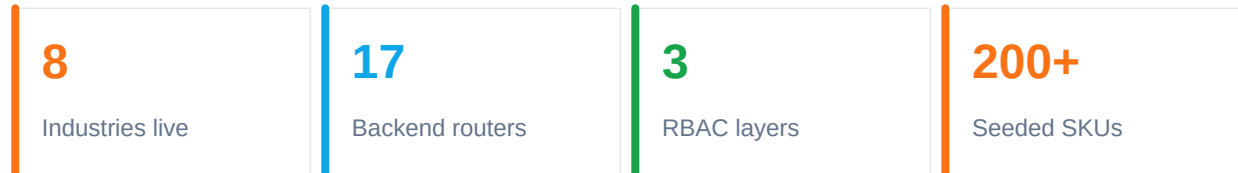
Retail • F&V • Restaurant • Cafe • Textile • Pharmacy • Hardware • Electronics

Contents

01	Executive Summary	3
02	Product Vision & Positioning	4
03	Industries Supported	5
04	Feature Matrix	6
05	System Architecture	8
06	Multi-Tenancy & RBAC Model	10
07	Data Model (MongoDB)	12
08	Backend API Surface	14
09	Frontend Architecture	16
10	Industry Deep-Dives	17
11	Billing Engine & Side-Effects	20
12	Integrations	21
13	Deployment & Environments	22
14	Roadmap	23

01 Executive Summary

Sparkbills is a multi-tenant, multi-industry GST billing and ERP platform built for Indian SMBs. A single React + FastAPI + MongoDB codebase adapts its UI and workflows per industry via a Super-Admin controlled *industry lock*, so a Pharmacy tenant sees batch/expiry POS while a Textile tenant sees size-wise stock — without any code fork.



Why Sparkbills?

- **One product, eight industries** — retail, F&V, restaurant, cafe, textile, pharmacy, hardware, electronics.
- **India-first GST compliance** — CGST/SGST/IGST, HSN/SAC, GSTR-1 and GSTR-3B exports.
- **Additive UI architecture** — industry-specific screens layer on top of a shared core; nothing hidden, nothing forked.
- **Platform Console** — Super-Admin can provision tenants, lock industries, impersonate for support, and audit every action.
- **Deep per-industry logic** — FIFO batch consumption (pharmacy), size-wise stock (textile), auto-warranty creation (electronics), delivery challans, KOT/waiter (restaurant).

02 Product Vision & Positioning

Vision

Give every Indian shopkeeper the same billing intelligence a corporate enterprise ERP offers — without the enterprise price tag, complexity, or long onboarding. Sparkbills replaces Tally-style desktop tools with a cloud-native, mobile-friendly, role-aware SaaS that speaks the specific language of each trade.

Target Customer

- Single-store and small-chain retailers (up to 10 branches).
- Restaurants, cafes and QSRs that need floor / KOT / waiter workflows.
- Regulated verticals — pharmacies (batch/expiry), electronics (warranty/serial).
- Wholesalers and hardware traders using quotation → sales-order → invoice pipeline.

Positioning Statement

For Indian SMBs across 8 trades, *Sparkbills* is the only GST-billing SaaS that adapts its POS, inventory and reports to the actual shape of your business — because a pharmacy is not a textile shop, and a restaurant is not a retail store.

Explicitly Out of Scope

- AI / GenAI features (no chatbots, no LLM copilots inside billing).
- HR / Payroll management.
- Manufacturing / MRP / BOM planning.

03 Industries Supported

Each tenant is **locked** to one or more industries at provisioning time. The industry set controls which sidebar modules, POS screens, seed catalogues and reports the tenant sees.

Industry	Core Workflow Highlights
Retail	General store, HSN-based GST billing, barcode POS, ledger.
Fruits & Vegetables	Weight/loose SKUs, bulk price grid, spoilage tracking, day-price sync.
Restaurant	Floor plan, tables, KOT, waiter board, split bills, kitchen printers.
Cafe	Fast-order POS, add-ons/modifiers, day-close, simpler than full restaurant.
Textile	Single-product model with size-wise stock , alterations kanban, brand/color/size POS.
Pharmacy	Batch/expiry, FIFO consumption , manufacturer reports, medicine catalog, prescription capture.
Hardware	Quotation → Sales Order → Invoice pipeline (Q-YYYYMM / SO-YYYYMM counters), 58-SKU seed.
Electronics	Deliveries kanban, warranties CRUD, auto-24-month warranty from invoice serials, delivery challan PDFs.

Removed by product decision: hospital, service, generic (superseded by the eight verticals above).

04 Feature Matrix

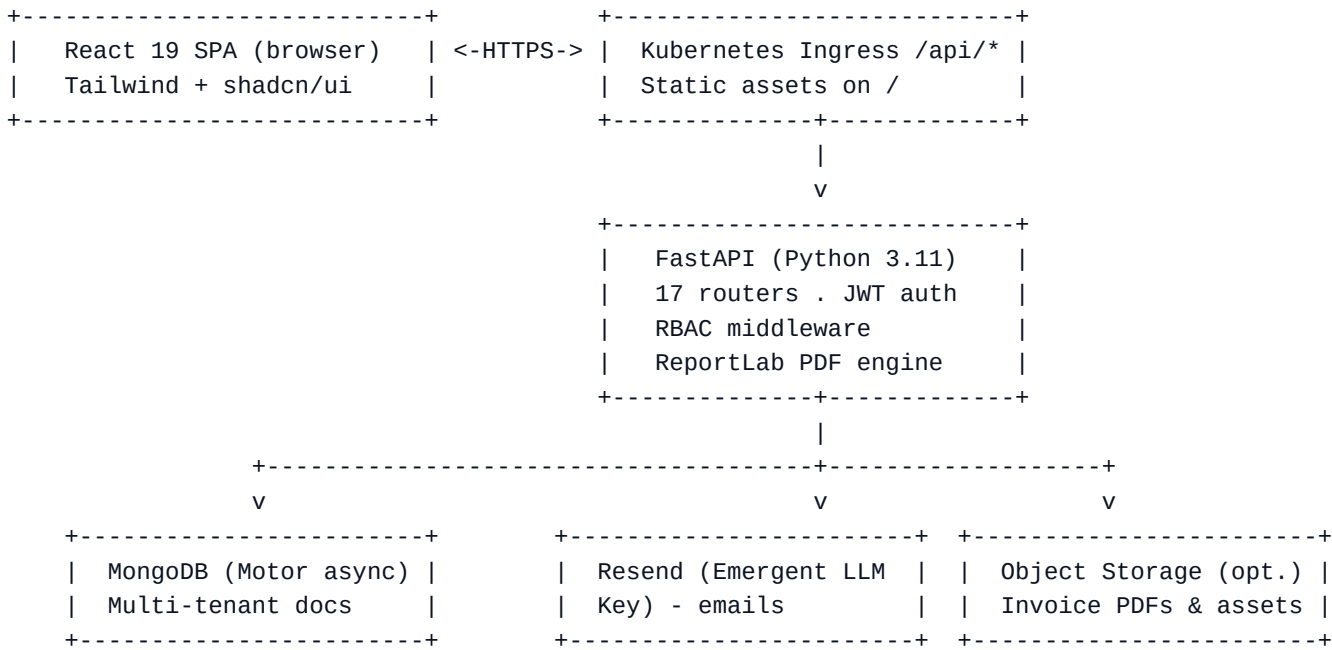
Feature	Retail	F&V	Rest.	Textile	Pharma	Hard.	Elec.
GST Invoices (CGST/SGST/IGST)	●	●	●	●	●	●	●
POS Screen	●	●	●	●	●	—	●
HSN / SAC Master	●	●	●	●	●	●	●
Batch & Expiry	—	—	—	—	●	—	—
Size-wise Stock	—	—	—	●	—	—	—
Kitchen Order Tickets (KOT)	—	—	●	—	—	—	—
Floor / Table Plan	—	—	●	—	—	—	—
Bulk Price Grid	—	●	—	—	—	—	—
Quotation → SO Pipeline	—	—	—	—	—	●	●
Deliveries Kanban	—	—	—	—	—	—	●
Warranties CRUD + Auto-Create	—	—	—	—	—	—	●
GSTR-1 / GSTR-3B Reports	●	●	●	●	●	●	●
Shifts / Cash Drawer	●	●	●	●	●	●	●
Multi-Branch	●	●	●	●	●	●	●
Impersonation Audit	●	●	●	●	●	●	●

● supported natively — not applicable to this industry

05 System Architecture

Sparkbills follows a classic **three-tier** layout with a stateless FastAPI app in the middle and a **MongoDB** document store. All traffic between the browser and the API is proxied through Kubernetes ingress; all data is scoped by `business_id`.

High-Level Diagram



Layer Responsibilities

Presentation	React 19 SPA, React-Router 7, shadcn/ui, Tailwind. Industry-aware routing and sidebar.
API	FastAPI with 17 routers, all endpoints prefixed <code><code>/api/</code></code> , Pydantic validation, RBAC g
Domain Logic	Billing engine with side-effects (stock decrement, FIFO batches, auto-warranty, deliveries).
Persistence	MongoDB via Motor async driver. Documents are UUID-keyed, tenant-scoped, month-scoped c
Reporting	ReportLab generates invoice / challan / warranty-card PDFs on demand.
Delivery	Kubernetes-hosted supervisor-managed processes; frontend at :3000, backend at :8001.

Request Lifecycle Example — POST /api/billing/invoices

1. Browser sends invoice payload -> Ingress routes /api/* to backend :8001
2. FastAPI JWT middleware verifies token -> attaches user + business_id
3. RBAC guard checks module_permissions['billing.write']
4. Route validates Pydantic InvoiceIn schema
5. Domain service:
 - decrements item stock (size-wise if textile)
 - FIFO-consumes batches (if pharmacy)
 - auto-creates warranties for lines with serial_no (if electronics)
 - upserts customer ledger entry
6. MySQL INSERT -> invoice row with month-scoped number INV-YYYYMM-NNNN
7. ReportLab renders A4 PDF, streamed back to client
8. Emit audit_log row {actor, action=invoice.create, entity_id}

06 Multi-Tenancy & RBAC Model

Sparkbills isolates tenants at the row level. Every collection carries `business_id`; queries always inject it from the JWT claim. There is no shared inventory or ledger between tenants.

Three RBAC Layers

Layer 0 — Industry	Super-Admin locks each tenant to a set of allowed industries (e.g. Pharmacy + Retail). Sidebar
Layer 1 — Module	Owner assigns modules per role (billing, items, reports, deliveries, warranties...). Stored on <code>
Layer 2 — Permission	Fine-grained: <code>invoice.create</code>, <code>invoice.void</code>, <code>stock.transfer</code>

Role Catalogue

Role	Scope	Default Access
super_admin	Platform	All tenants, impersonation, provisioning, industry lock.
owner	Tenant	Full access inside the tenant.
manager	Tenant	All operational modules, no billing/settings destructive ops.
cashier	Tenant	POS + shift close only.
waiter	Restaurant	KOT + table board, no reports.
kitchen	Restaurant	KOT display, mark ready.
salesperson	Electronics	POS + deliveries + warranties.
technician	Electronics	Deliveries + warranties only.
warehouse	Electronics	Items + stock ops + purchases + deliveries.
accountant	Tenant	Reports, ledger, GSTR read-only.

Impersonation & Audit

- Super-Admin can impersonate any tenant user; a red banner appears in the UI.
- Every impersonation session is logged with actor, target, start/stop, and IP.
- All mutating routes emit audit_log rows visible in Platform Console.

Tenant Provisioning Flow

Super-Admin -> Platform Console -> Tenants -> New Tenant

```
|-- Business name, GSTIN, state
|-- Allowed industries [ retail | pharmacy | electronics ... ] <- Layer 0 lock
|-- Owner email + temp password (mailed via Resend)
+-- Seed catalogue (optional) - POST /api/items/seed/<industry>
```

07 Data Model (MongoDB)

Every document has id (UUID string), business_id, created_at, updated_at. Numbers like invoice codes use an atomic counter keyed by (business_id, prefix, YYYYMM).

Collection	Key fields
businesses	name, gstin, state, allowed_industries[], modules{}, settings{}
users	email, password_hash, role, platform_role, module_permissions{}, business_id
items	sku, name, hsn, price, tax_rate, size_stocks[{size,qty}], batches[{batch_no, expiry, qty}], warranty_months, model_number, imei_required
customers	name, phone, gstin, state, ledger_balance
suppliers	name, phone, gstin, opening_balance, pending_amount
invoices	number, business_id, customer_id, lines[], totals{cgst,sgst,igst,round_off}, payment_mode, shift_id
purchases	number, supplier_id, lines[], totals, batch_data
quotations	number Q-YYYYMM-NNNN, customer, valid_till, lines[]
sales_orders	number SO-YYYYMM-NNNN, converted_from quotation_id, status
shifts	cashier_id, opened_at, closed_at, opening_cash, closing_cash, variance
kots	table_id, waiter_id, items[], status: new/preparing/ready/served
tables	floor_id, name, capacity, status
deliveries	number DLV-YYYYMM-NNNN, invoice_id, customer, address, slot, technician, status, items[]
warranties	serial_no, imei, brand, model, customer, start_date, end_date, months, status
alterations	customer, garment, size, tailor, status kanban
vouchers	type: receipt/payment/journal/contra, party, amount, ref
audit_logs	actor_id, actor_email, action, entity, entity_id, before, after, ts

Collection	Key fields
impersonation_sessions	super_admin_id, target_user_id, started_at, ended_at, ip

08 Backend API Surface

All endpoints are prefixed /api and grouped under 17 routers.

Router	Representative Endpoints
auth	/api/auth/login, /api/auth/me, /api/auth/refresh
masters	/api/items, /api/customers, /api/suppliers, /api/branches
billing	/api/billing/invoices, /api/billing/void, /api/billing/pdf/{id}
purchase	/api/purchases, /api/purchases/import
inventory	/api/inventory/adjust, /api/inventory/transfer, /api/inventory/lowstock
restaurant	/api/restaurant/tables, /api/restaurant/kots, /api/restaurant/floors
textile	/api/textile/alterations, /api/textile/size-variants
pharmacy	/api/pharmacy/batches, /api/pharmacy/manufacturer-report
electronics	/api/deliveries, /api/warranties, /api/electronics/dashboard, /api/electronics/departments
hardware_pipeline	/api/quotations, /api/sales-orders, /api/hardware/convert
reports	/api/reports/gstr1, /api/reports/gstr3b, /api/reports/sales, /api/reports/profitability
shifts	/api/shifts/open, /api/shifts/close, /api/shifts/recent
vouchers	/api/vouchers, /api/vouchers/{id}
settings	/api/settings, /api/settings/branding, /api/settings/pdf
notifications	/api/notifications, /api/notifications/mark-read
webhooks	/api/webhooks/inbound, /api/webhooks/emit
platform	/api/platform/tenants, /api/platform/impersonate, /api/platform/audit

09 Frontend Architecture

The SPA is organized around two shells: AppLayout for tenant users and PlatformLayout for Super-Admins. Industry gating happens at route registration time, driven by `business.allowed_industries`.

Directory Layout

```
frontend/src
+-- layouts/
|   |-- AppLayout.jsx           # tenant shell (topbar + sidebar + outlet)
|   +-- PlatformLayout.jsx      # super-admin shell
+-- pages/
|   |-- Dashboard.jsx           # per-industry dashboards
|   |-- POS.jsx / NewInvoice.jsx
|   |-- Items.jsx Customers.jsx Suppliers.jsx Ledger.jsx
|   |-- Restaurant.jsx (~1000 lines - flagged for refactor)
|   |-- Pharmacy.jsx Electronics.jsx
|   |-- Deliveries.jsx Warranties.jsx Alterations.jsx
|   |-- Reports.jsx Shift.jsx StockOps.jsx Vouchers.jsx
|   |-- PlatformDashboard.jsx PlatformTenants.jsx PlatformImpersonationAudit.jsx
|   +-- Login.jsx Register.jsx Landing.jsx
+-- components/
|   |-- ui/                     # shadcn primitives (Button, Dialog, Table)
|   |-- PermissionsManager.jsx
|   |-- IndustryChip.jsx
|   +-- InvoicePDFPreview.jsx
+-- lib/ api.js auth.js format.js industries.js
```

State & Data Fetching

- Auth token stored in `localStorage`; injected via Axios interceptor.
- Server state via `react-query`-style hooks (`fetch`, `invalidate` on mutation).
- Toasts through `sonner`; forms via `react-hook-form`.
- Charts via `Recharts` (sales trends, GSTR filings, delivery flow).

10 Industry Deep-Dives

Textile

67-SKU seed with a **single-product model** where each item carries a `size_stocks[]` array — e.g. one shirt SKU with quantities per S/M/L/XL. The POS shows a size picker and the invoice decrements only the chosen size. An Alterations Kanban tracks tailor jobs (received → in-progress → ready → delivered).

Pharmacy

48-medicine seed across 14 categories. Every item holds a `batches[]` array with `batch_no`, `mfg`, `expiry`, `qty`. POS auto-selects the earliest-expiry batch (**FIFO**), decrements it, and rolls to the next when depleted. Reports include manufacturer-wise sales and expiring-in-90-days.

Electronics & Home Appliances

37-SKU seed across 8 departments (TV, Fridge, Washing, AC, Kitchen, Computers, Accessories, CCTV). Every invoice line with a serial number auto-creates a 24-month **warranty** row. Deliveries page shows a 6-column Kanban (all → scheduled → out_for_delivery → delivered → installed → cancelled) with technician assignment and installation slots. PDF **delivery challans** and A6 warranty cards are generated on demand.

Restaurant & Cafe

Floor-plan editor with tables and capacity, a waiter board with drag-to-assign, KOT flow with kitchen-side status, split billing per table, and menu modifiers. Cafe mode is a lightweight variant without floor plan for counter-service outlets.

Hardware & Wholesale

58-SKU seed and a **quotation** → **sales-order** → **invoice** pipeline with month-scoped counters (Q-YYYYMM-NNNN, SO-YYYYMM-NNNN). Bulk-discount tiers and customer-specific price lists.

Fruits & Vegetables

Weight-based SKUs, spoilage adjustments, and a planned *Bulk Price Update Grid* so a shopkeeper can retype the day's mandi prices for 40 items in one pane.

11 Billing Engine & Side-Effects

The billing router is the busiest module — every industry converges here at checkout. Side-effects are dispatched from a single service layer so behaviour stays consistent across POS, restaurant, and back-office invoice screens.

Side-Effect Matrix

Side-effect	Trigger & scope
Stock decrement	All industries — reduces item / size-variant / batch qty.
FIFO batch pick	Pharmacy — earliest expiry first, roll over on depletion.
Size-wise decrement	Textile — decrements size_stocks[i].qty for chosen size.
Warranty auto-create	Electronics — creates warranties row for each serial line.
Ledger entry	Customer receivable / cash / bank ledger updated atomically.
GST accumulators	CGST/SGST/IGST buckets updated for GSTR-1/3B reports.
Counter increment	Atomic month-scoped invoice number INV-YYYYMM-NNNN.
Notification emit	Low-stock, expiring-warranty, refill-due events queued for tenant.
Audit log	actor + before/after diff written to audit_logs.

12 Integrations

Integration		Purpose & notes
Resend (Email)		Owner welcome emails, invoice PDF share, expiring-warranty reminders. Uses the Emergent LLM Key managed by the platform — tenants never see an API key.
MongoDB		Primary datastore, accessed via Motor async driver. Multi-tenant row-level isolation on business_id.
ReportLab		In-process PDF generation for invoices, quotations, delivery challans, warranty cards, GSTR export cover-sheets.
Kubernetes Supervisor	+	Backend (:8001), frontend (:3000), and MongoDB run as supervisor programs inside a Kubernetes pod; ingress routes /api/*.
Object Storage (optional)		For persistent PDF archive and branding assets. Wired via Emergent object-storage playbook when tenant opts in.

13 Deployment & Environments

Frontend	React 19 build served at port 3000 (dev) / static bundle in prod. Uses <code>REACT_APP_BACKEND_URL</code> .
Backend	FastAPI + Uvicorn under Supervisor at 0.0.0.0:8001. All routes prefixed <code>/api/</code> .
Database	MongoDB via <code>MONGO_URL</code> , database name from <code>DB_NAME</code> .
Ingress	Kubernetes ingress: <code>/api/*</code> → backend :8001; <code>/</code> → frontend :3000.
Hot Reload	Frontend + backend auto-reload on file changes; supervisor restart only for <code>.env</code> / deps.
Secrets	Environment variables only, never checked in. Emergent LLM Key auto-injected for AI/Resend.
Backups	MongoDB daily snapshot; per-tenant export via <code>/api/settings/backup</code> .

Sizing (default pod)

- 1 vCPU / 2 GB RAM comfortably serves ~50 concurrent tenant users.
- Mongo working set stays < 500 MB for a mid-size tenant (1 year of data).
- PDF generation is CPU-bound; can be offloaded to a worker pool if needed.

14 Roadmap

Delivered (Feb 2026)

- Textile industry — single-product size-wise stock + Alterations Kanban.
- Pharmacy industry — batch/expiry, FIFO POS, manufacturer report backend.
- Electronics industry — Deliveries Kanban, Warranties CRUD, auto-warranty from invoice.
- Delivery Challan PDF via ReportLab.
- Multi-tenant industry lock & Platform Console impersonation audit.

P1 — Next Up

- Shift-required-for-POS lockout (cashier/waiter cannot bill without an open shift).
- Bulk Price Update Grid — spreadsheet-style daily price editor for F&V.
- CSV export across Ledger, Recent Shifts, GSTR, Reports.
- Quick Add Staff wizard with auto-password + copyable login link.
- Manufacturer / Medicine sales report **UI** page (endpoints already live).
- Auto-print A6 warranty card PDF with QR link back to /warranties?serial=....

P2 — Backlog

- Refactor Restaurant.jsx (~1000 lines) into FloorPlanTab / WaiterBoardTab / PosTab.
- Recurring invoices & subscription billing.
- Offline PWA POS (queue & sync).
- Table QR menu with auto-KOT.
- In-app product tour ("see it in 60 seconds") modal.
- "Powered by Sparkbills" PDF watermark for free tier.
- WhatsApp reminders — tailor-ready, warranty expiring, refill due.
- IMEI barcode scanning on POS & delivery route optimization.

Explicitly Rejected

- AI / GenAI features inside billing.
- HR / Payroll modules.
- Manufacturing / BOM planning.